

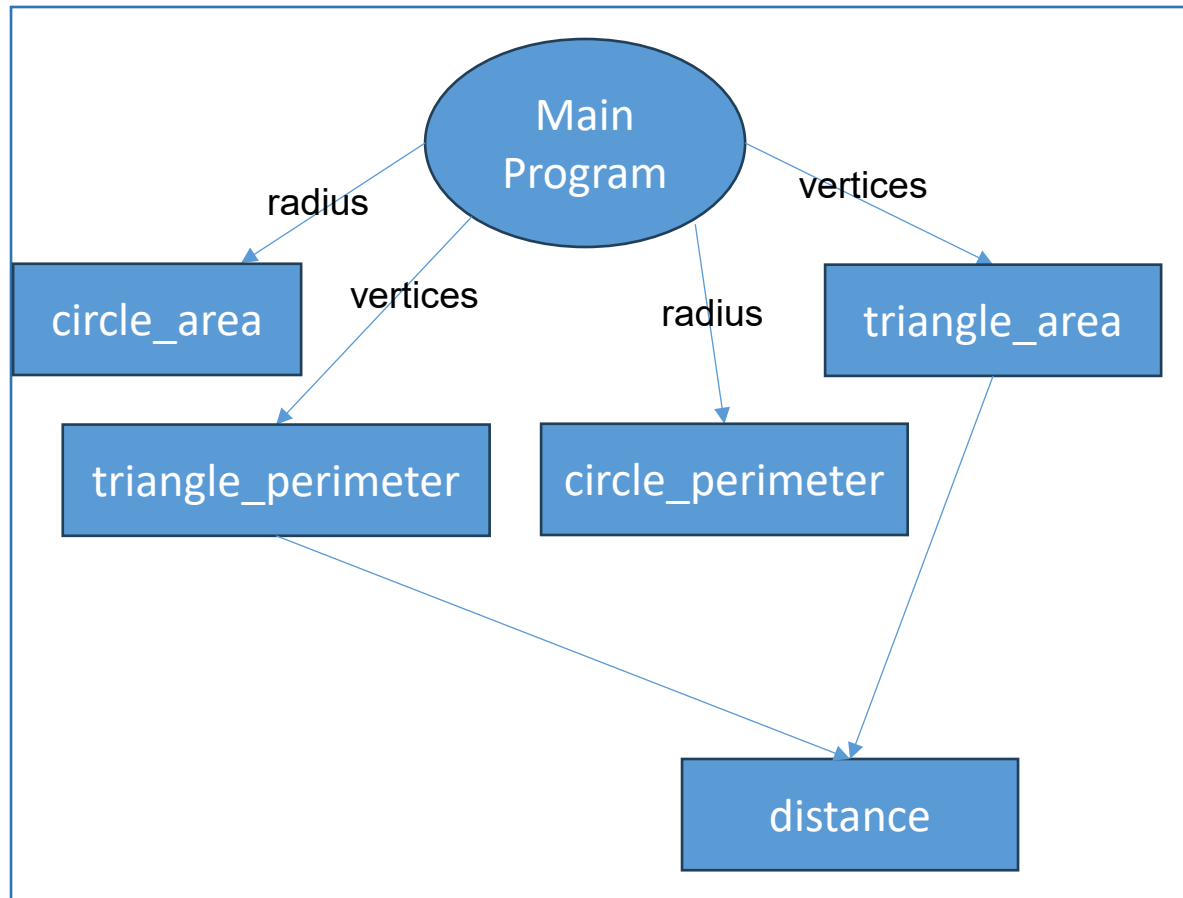
Fundamentals of Programming & Computer Science CS 15-112

OOP – Part 1

Hend Gedawy

The Idea Behind OOP

What we have been doing so far



```
# Function to calculate area of circle
def calculate_circle_area(radius):
    return 3.14 * radius * radius

# Function to calculate perimeter of circle
def calculate_circle_perimeter(radius):
    return 2 * 3.14 * radius

# Function to calculate the distance between two points in a 2D plane.
def distance(point1, point2):
    x1, y1 = point1
    x2, y2 = point2
    return math.sqrt((x2 - x1)**2 + (y2 - y1)**2)

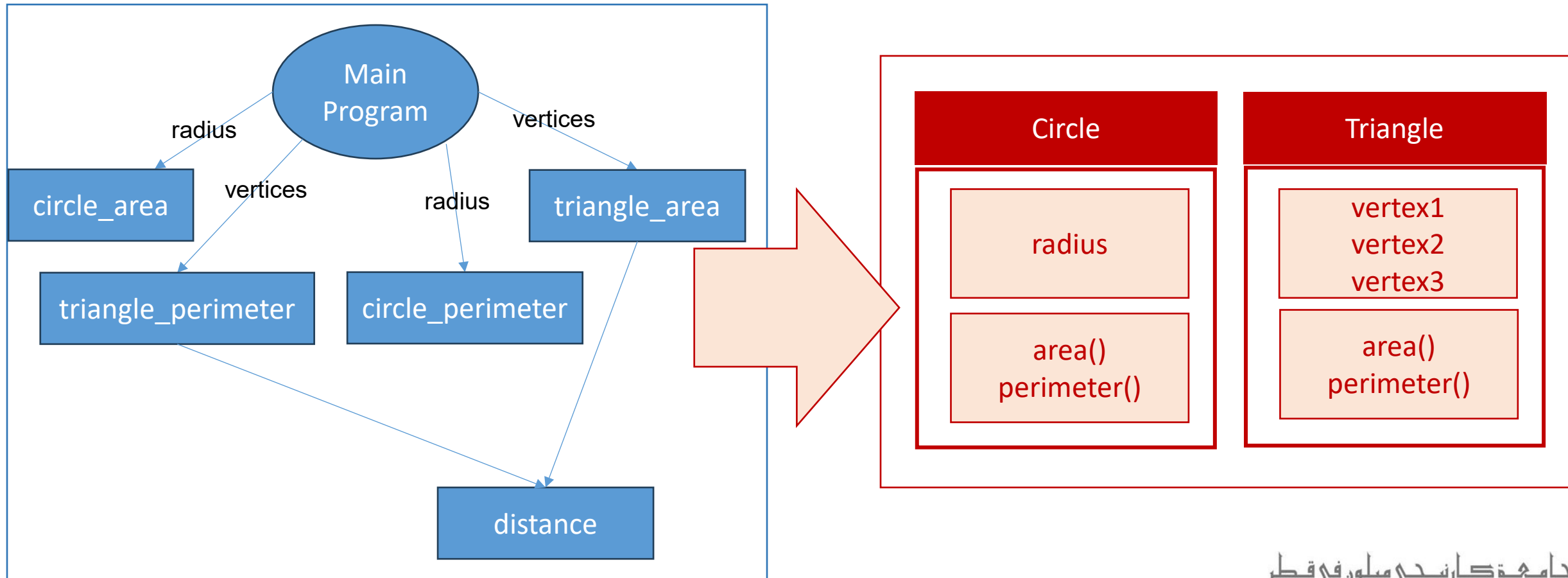
# Function to calculate area of triangle
def calculate_triangle_area(side1, side2, side3):
    side1 = distance(vertices[0], vertices[1])
    side2 = distance(vertices[1], vertices[2])
    side3 = distance(vertices[2], vertices[0])
    s = (side1 + side2 + side3) / 2 # Semi-perimeter
    area = math.sqrt(s * (s - side1) * (s - side2) * (s - side3))
    return area

# Function to calculate perimeter of triangle
def calculate_triangle_perimeter(vertices):
    side1 = distance(vertices[0], vertices[1])
    side2 = distance(vertices[1], vertices[2])
    side3 = distance(vertices[2], vertices[0])
    perimeter = side1 + side2 + side3
    return perimeter

# Main Program
circle_perimeter = calculate_circle_perimeter(3)
circle_area = calculate_circle_area(3)

vertices = [(0, 0), (3, 4), (6, 0)]
triangle_area = calculate_triangle_area(vertices)
triangle_perimeter = calculate_triangle_perimeter(vertices)
```

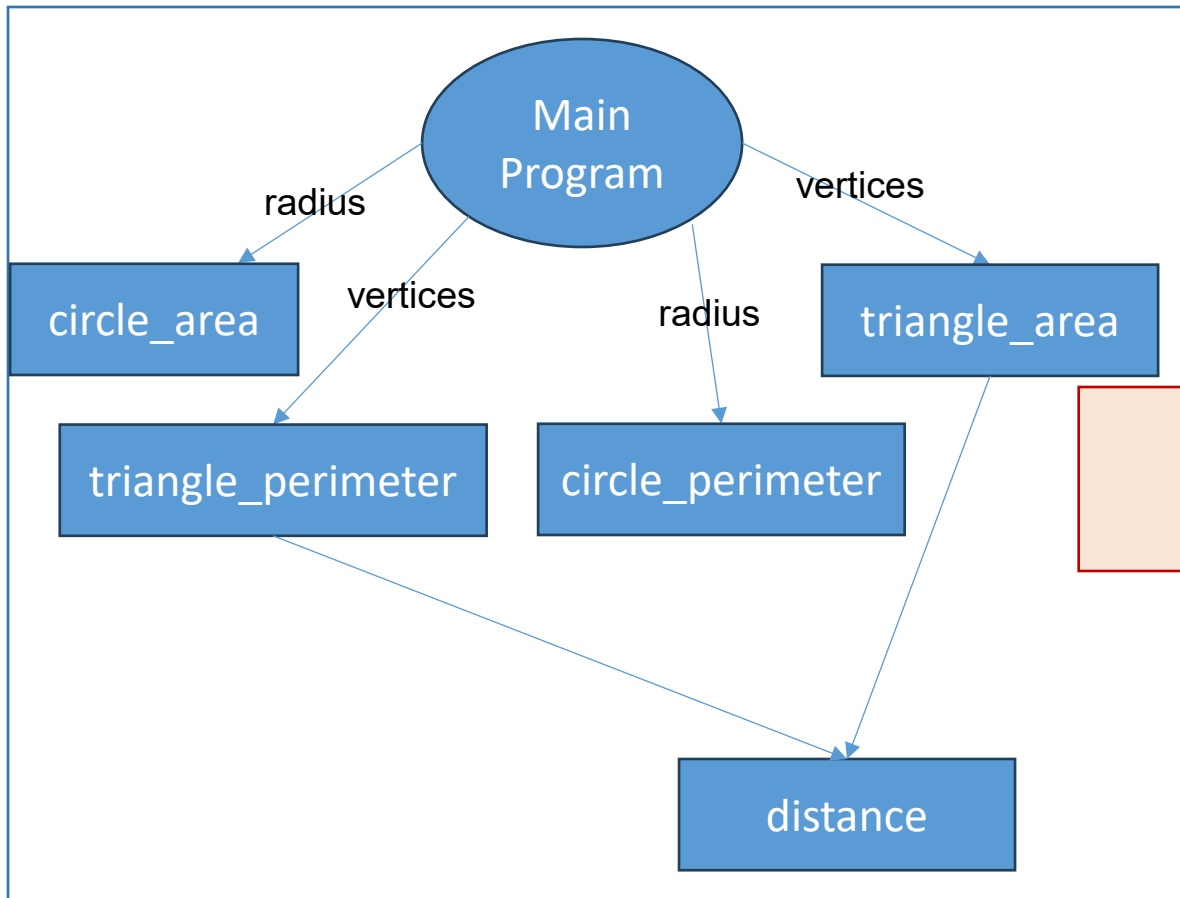
An Alternative Approach



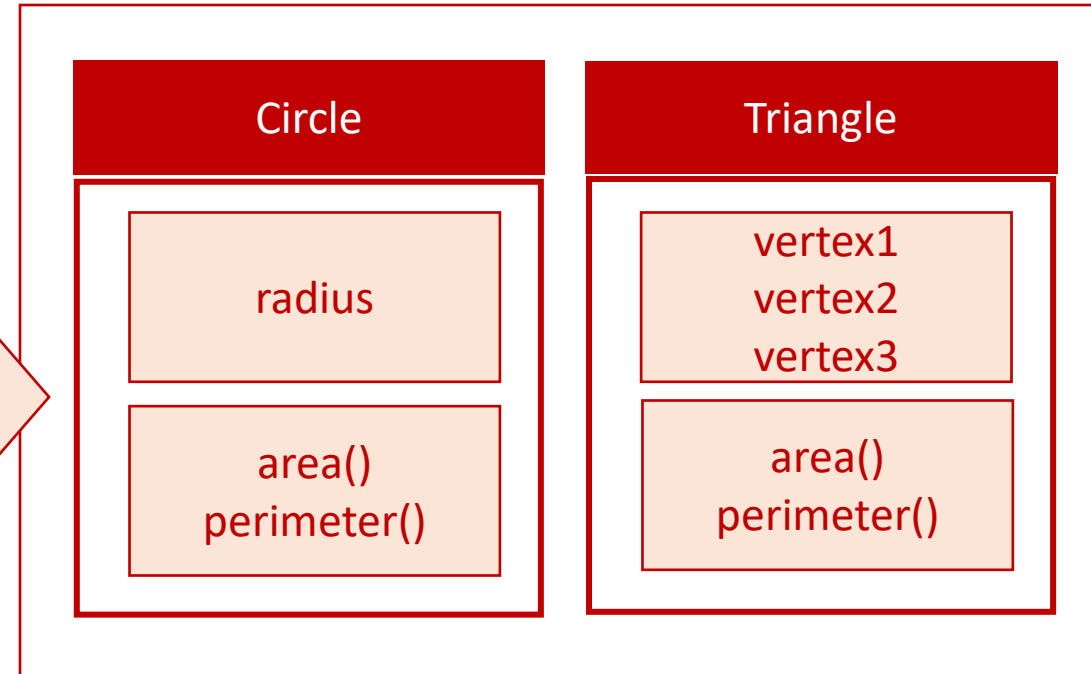
An Alternative Approach

Your program is divided into parts

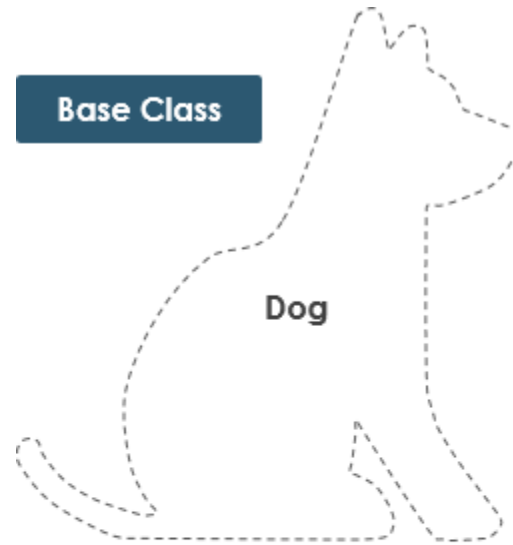
Functions



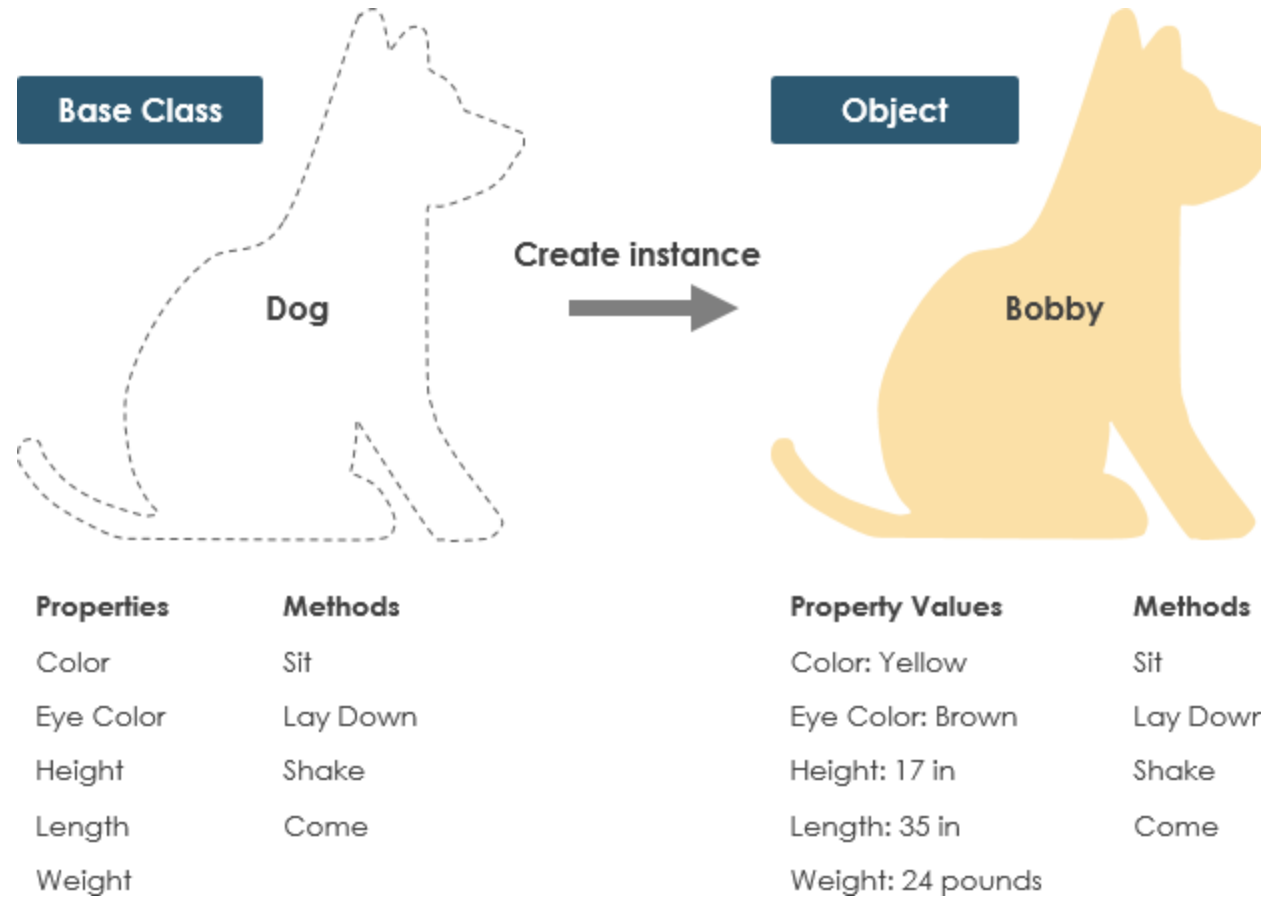
Classes



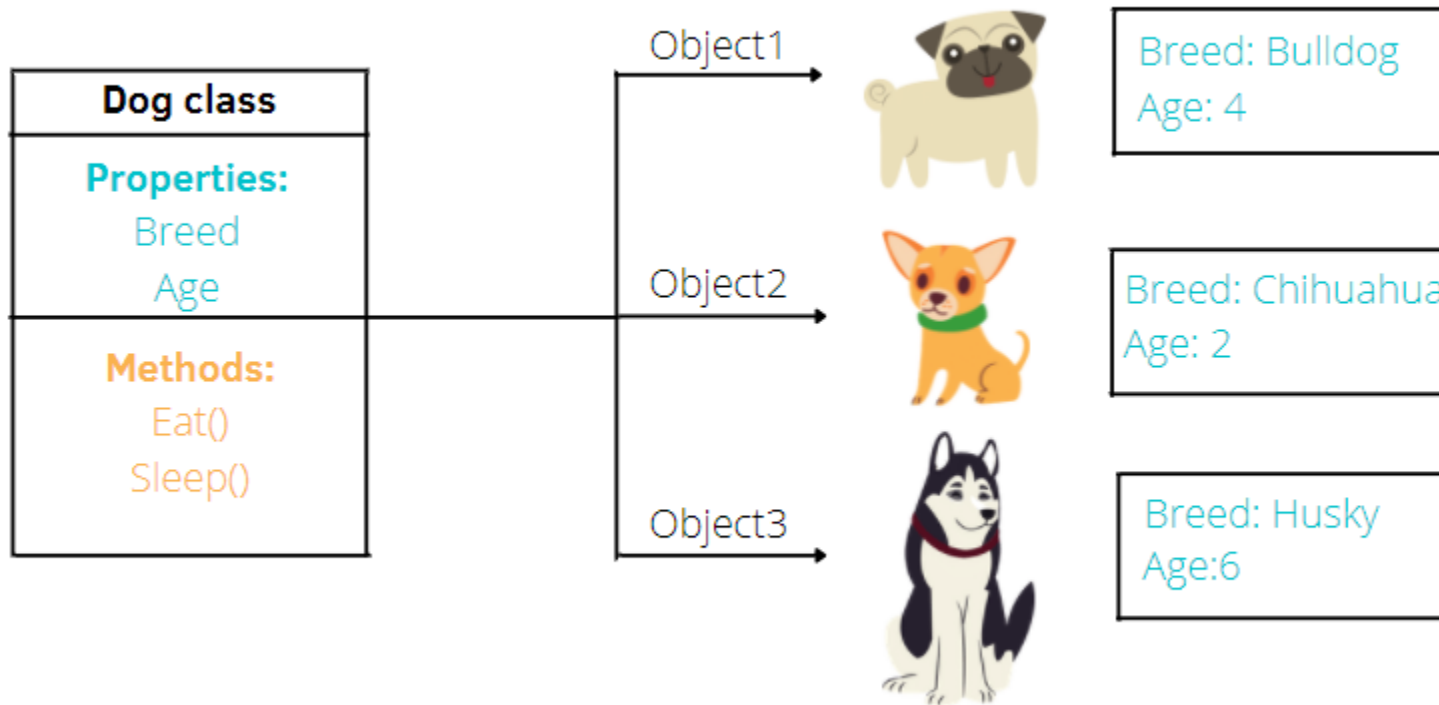
Object Oriented Programming (OOP) Approach



Classes _ Objects/Instances



Classes _ Objects/Instances



Acts as a template for
a generic object.

Instances

Did you
know

you have already been using OOP

You have been using Built-in Objects in Python

Class

Object/Instance

str		"Hello"
int		5
list		[1,2,3,4]

You have been using Built-in Objects in Python

Class

(are also Types)

Object/Instance

(values of a given class or type)

str



"Hello"

An instance of type **str**

```
>>> type("Hello") == str
True
>>> isinstance("Hello", str)
True
```

int



5

An instance of type **int**

```
>>> type(5) == int
True
>>> isinstance(5, int)
True
```

list



[1,2,3,4]

An instance of type **list**

```
>>> type([1,2,3,4]) == list
True
>>> isinstance([1,2,3,4], list)
True
```

You have been calling methods on these objects

```
s = 'This could be any string!'
```

We call methods using s.method()...

```
print(s.upper()) # upper is a string method, called using the . Notation  
# we say that we "call the method upper on the string s"
```

```
print(s.replace('could', 'may')) # some methods take additional  
arguments
```

... rather than function(s):

```
print(len(s)) # len is a function
```

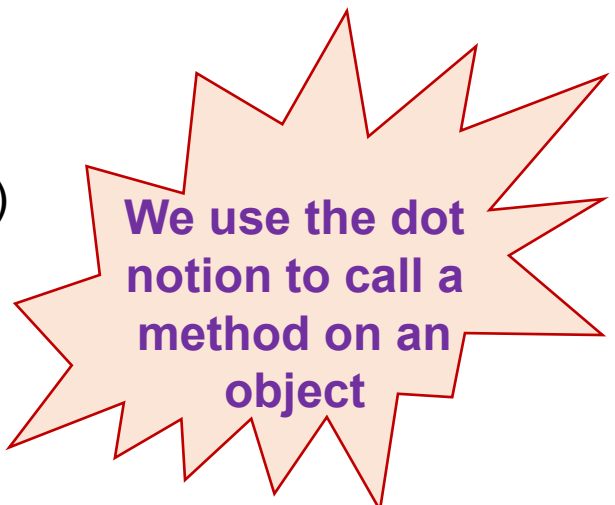
Functions Versus Methods

f(_) is a Function

len(L)
sorted(L)
max(n1, n2, n3..)
abs(n)
range(n)
sum(L)

obj.m(_) is a Method

s.upper()
l.append(x)
d.clear()
l.count(x)
s1.update(s2)
l.sort()



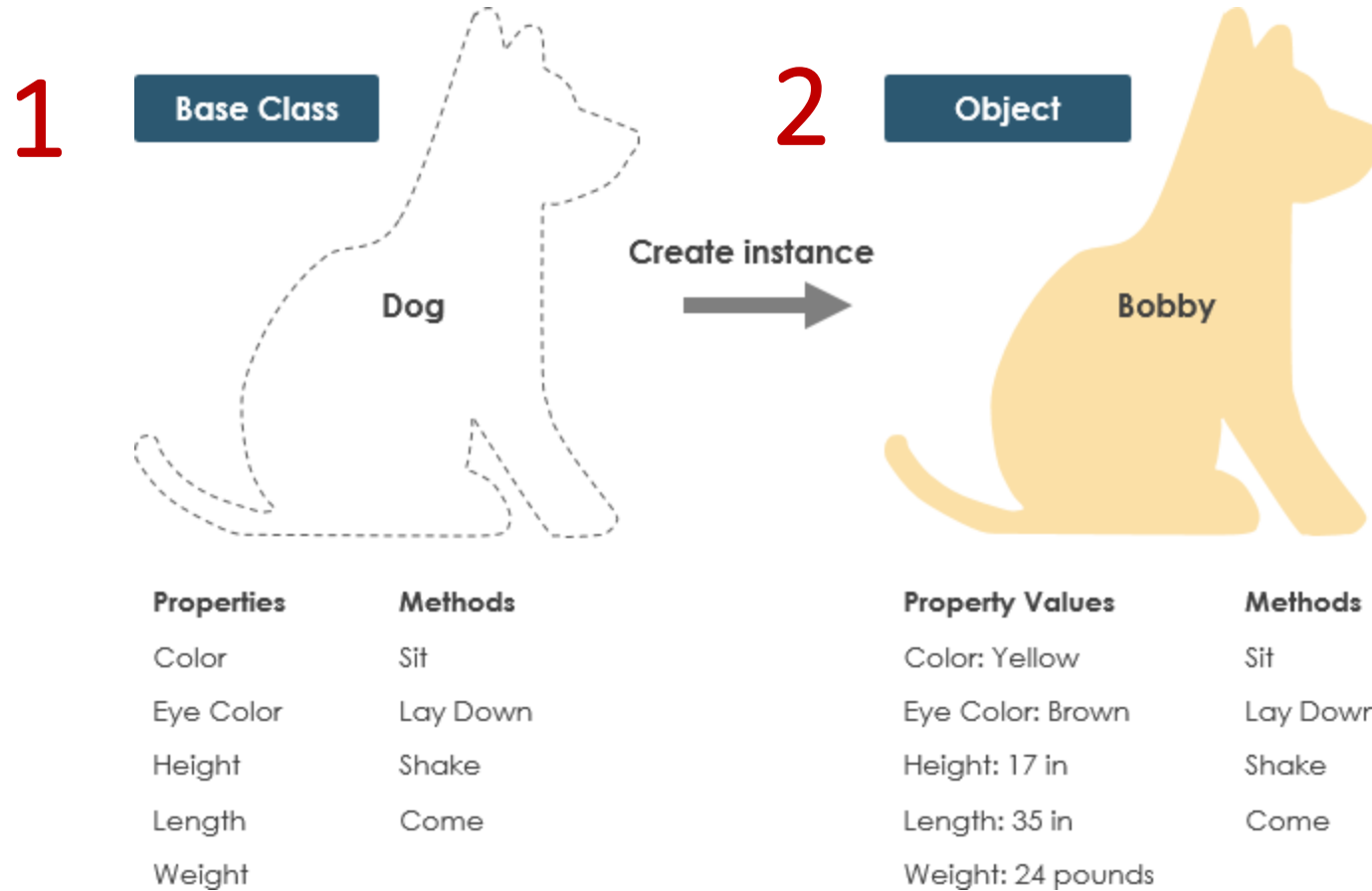
We use the dot
notation to call a
method on an
object

≡ create ≡
your own
Objects

Creating Your Own Objects

- **Properties?**
 - What data defines your Object?
- **Methods?**
 - How do you plan to use the object in your code?

Creating Your Own Objects



Creating Empty Class & Instances

Create your own class:

```
class Dog(object):  
    # define properties and methods of a generic dog here  
  
    # a class must have a body, even if it does nothing, so we will  
    # use 'pass' for now...  
    pass
```

Create instances of our class:

```
d1 = Dog()  
d2 = Dog()
```

Verify the type of these instances:

```
print(type(d1))    # Dog (actually, class '__main__.Dog')  
print(isinstance(d2, Dog)) # True
```

```
# Create your own class:
class Dog(object):
    # define properties and methods of a
    generic dog here

    # a class must have a body, even if it does nothing, so we
    will
    # use 'pass' for now...
    pass

# Create instances of our class:
d1 = Dog()
d2 = Dog()

# Set and get properties (aka 'fields' or 'attributes') of these instances:
d1.name = 'Dot'
d1.age = 4
d2.name = 'Elf'
d2.age = 3
print(d1.name, d1.age) # Dot 4
print(d2.name, d2.age) # Elf 3
```

Properties

Setting and Getting

Instances Properties

```
def constructor(dog, name, age):  
    # pre-load the dog instance with the given name and age:  
    dog.name = name  
    dog.age = age
```

```
def __init__(dog, name, age):  
    # pre-load the dog instance with the given name and age:  
    dog.name = name  
    dog.age = age
```

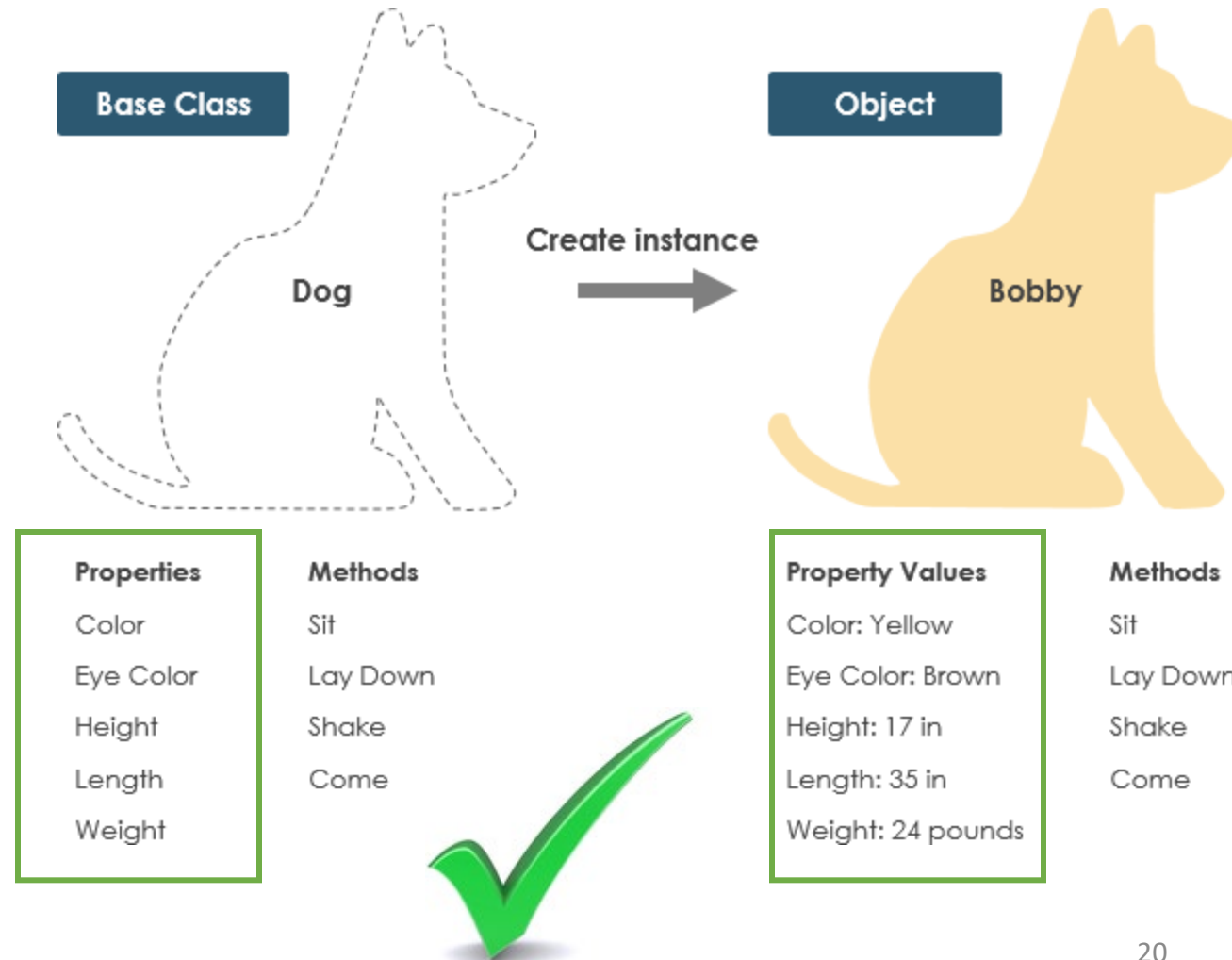
```
class Dog(object):  
    def __init__(self, name, age):  
        # pre-load the dog instance with the given name and age:  
        self.name = name  
        self.age = age
```

```
d1 = Dog('fred', 4) # now d1 is a Dog instance with name 'fred' and age 4  
d2 = Dog('Elf', 3)  
print(d1.name, d1.age) # Dot 4  
print(d2.name, d2.age) # Elf 3
```

Properties

Preloading Instances w/ Properties (using Constructors)

Creating Your Own Objects



```
class Dog(object):
```

```
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

Here is a function we will turn into a method:

```
def sayHi(dog):  
    print(f'Hi, my name is {dog.name} and I  
am {dog.age} years old!')
```

```
d1 = Dog('Dot', 4)  
d2 = Dog('Elf', 3)  
sayHi(d1) # Hi, my name is Dot and I am 4 years old!  
sayHi(d2) # Hi, my name is Elf and I am 3 years old!
```

Methods

1) Start with a Function

Methods

2) Turn it into Method

```
class Dog(object):
```

```
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

Now it is a method (simply by indenting it inside the class!)

```
    def sayHi(dog):  
        print(f'Hi, my name is {dog.name} and I am {dog.age} years old!')
```

```
d1 = Dog('Dot', 4)
```

```
d2 = Dog('Elf', 3)
```

Notice how we change the function calls into method calls:

```
d1.sayHi() # Hi, my name is Dot and I am 4 years old!
```

```
d2.sayHi() # Hi, my name is Elf and I am 3 years old!
```

Methods

2) Turn it into Method

```
class Dog(object):
```

```
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

Now it is a method (simply by indenting it inside the class!)

```
    def sayHi(dog):  
        print(f'Hi, my name is {dog.name} and I am {dog.age} years old!')
```

```
d1 = Dog('Dot', 4)
```

```
d2 = Dog('Elf', 3)
```

Notice how we change the function calls into method calls:

```
d1.sayHi() # Hi, my name is Dot and I am 4 years old!
```

```
d2.sayHi() # Hi, my name is Elf and I am 3 years old!
```

Methods

2) Turn it into Method

```
class Dog(object):
```

```
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

Use self keyword to represent the current instance

```
    def sayHi(self):  
        print(f'Hi, my name is {dog.name} and I am {dog.age} years old!')
```

```
d1 = Dog('Dot', 4)
```

```
d2 = Dog('Elf', 3)
```

```
d1.sayHi() # Hi, my name is Dot and I am 4 years old!
```

```
d2.sayHi() # Hi, my name is Elf and I am 3 years old!
```

More Methods

```
class Dog(object):
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
        self.woofCount = 0      # We initialize the property in the constructor!
```

```
    def sayHi(self):
```

```
        print(f'Hi, my name is {dog.name} and I am {dog.age} years old!')
```

```
# This method takes a second parameter -- times
```

```
    def bark(self, times):
```

```
        print(f'{self.name} says: {"woof!" * times}')
```

```
        self.woofCount += times # Then we can set and get the property in this method
```

```
d = Dog('Dot', 4)
```

```
d.sayHi() # Hi, my name is Dot and I am 4 years old!
```

```
d.bark(1) # Dot says: woof!
```

```
d.bark(4) # Dot says: woof!woof!woof!woof!
```



**Practice
Makes
Perfect**

Practice

Write the class SpiderMan to pass the test cases shown below.

Do not hardcode against the testcase values, though you can assume the testcases and comments cover the needed functionality.



```
s = SpiderMan("Peter")
# SpiderMan can shoot a web
assert(s.shootWeb() == "Peter shoots a web")
# SpiderMan can get hurt
assert(s.takeDamage(25) == "Peter gets hit for 25 hp")
```



What does the constructor take?

- What properties?

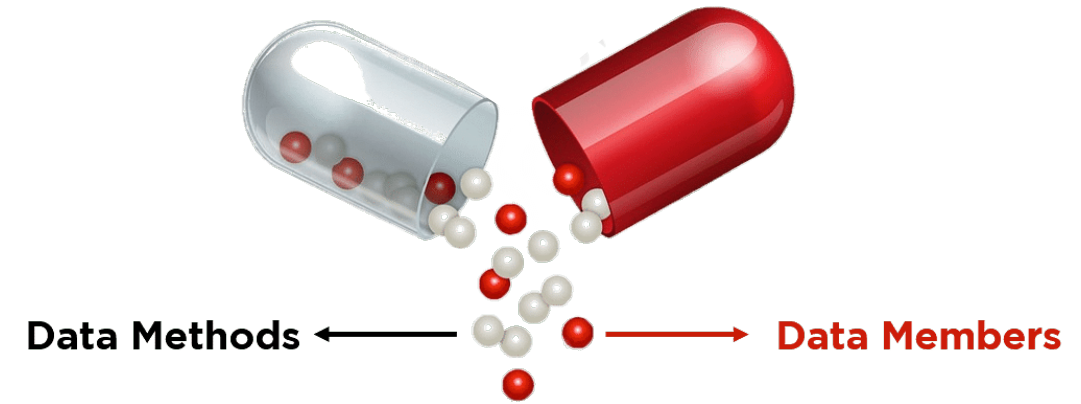
What methods is the code expecting you to implement ?

- What properties are these methods getting and setting/modifying?



Advantages of Classes and Methods

Encapsulation



- **Organizes code**

A class includes the data and methods for that class.

- **Promotes intuitive design**

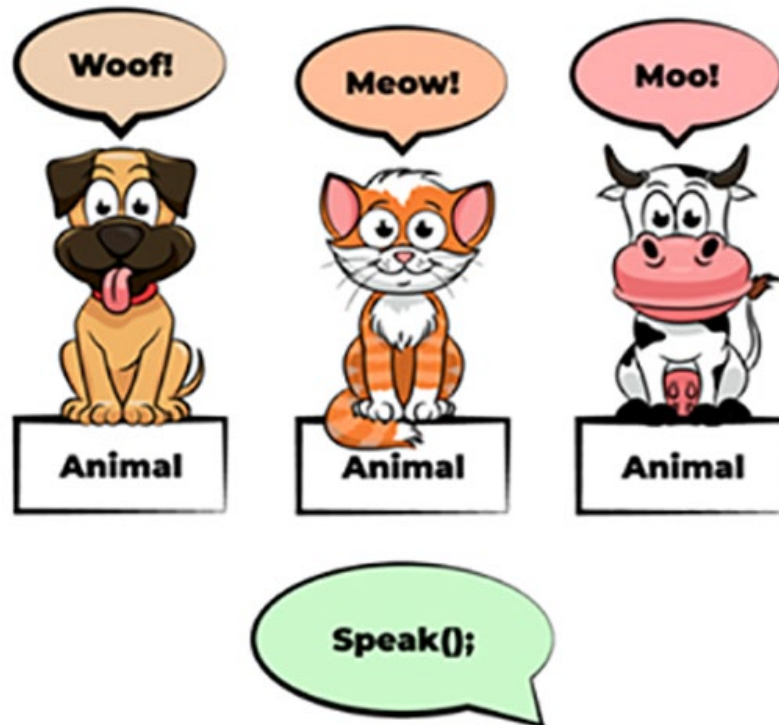
Well-designed classes should be *intuitive*, so the data and methods in the class match commonsense expectations.

- **Restricts access**

- `len` is a function, so we can call `len(True)` (which crashes)
- `upper` is a method on strings but not booleans, so we cannot even call `True.upper()`

Polymorphism

The same method name can run different code based on type.



```
class Dog(object):  
    def speak(self):  
        print('woof!')  
class Cat(object):  
    def speak(self):  
        print('meow!')  
  
for animal in [ Dog(), Cat() ]:  
    animal.speak() # same method name, but one  
                    woofs and one meows!
```

More

SPECIAL

Methods

Equality Testing

```
1 class A(object):  
2     def __init__(self, x):  
3         self.x = x  
4 a1 = A(5)  
5 a2 = A(5)  
6 print(a1 == a2)  # False!
```

Shouldn't a1= a2??

`__eq__` Equality Testing

The `__eq__` Method:

- Returns True if the object is equal to another object (**other**)
- Python uses it for testing equality of two objects

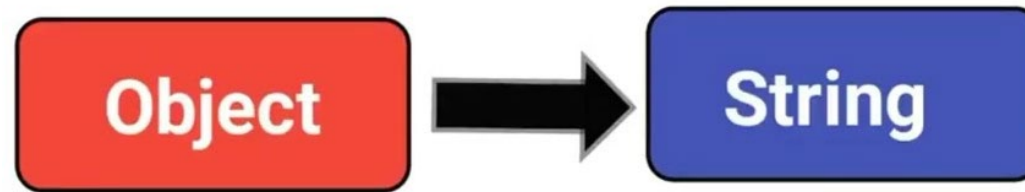
```
1 class A(object):
2     def __init__(self, x):
3         self.x = x
4     def __eq__(self, other):
5         return (self.x == other.x)
6 a1 = A(5)
7 a2 = A(5)
8 print(a1 == a2) # True
9 print(a1 == 99) # crash (darn!)
```

`__eq__` Equality Testing

```
1 class A(object):
2     def __init__(self, x):
3         self.x = x
4     def __eq__(self, other):
5         return (isinstance(other, A) and (self.x == other.x))
6 a1 = A(5)
7 a2 = A(5)
8 print(a1 == a2) # True
9 print(a1 == 99) # False (huzzah!)
```

Here we don't crash on unexpected types of `other`

Converting to String



```
1 class A(object):  
2     def __init__(self, x):  
3         self.x = x  
4 a = A(5)  
5 print(a) # prints <__main__.A object at 0x102916128> (yuck!)
```

Converting to String

`__str__`

```
1 class A(object):
2     def __init__(self, x):
3         self.x = x
4     def __str__(self):
5         return f'A(x={self.x})'
6 a = A(5)
7 print(a) # prints A(x=5) (better)
8 print([a]) # prints [<__main__.A object at 0x102136278>] (yuck!)
```

The `__str__` method tells Python how to convert the object to a string, but it is not used in some cases (such as when the object is in a list):

Converting to String

`__repr__`

The `__repr__` method is used inside lists (and other places)

```
1 class A(object):
2     def __init__(self, x):
3         self.x = x
4     def __repr__(self):
5         return f'A(x={self.x})'
6 a = A(5)
7 print(a) # prints A(x=5) (better)
8 print([a]) # [A(x=5)]
```

Recap: Some Special Methods

- `__init__`: constructor, initializer
 - Define initial values for the properties when an object is created
 - It does NOT return any meaningful value
- `__repr__`:
 - Returns a string
 - Python uses it to convert an object to a string
 - E.g., `print`
- `__eq__`: comparator
 - Returns True if the object is equal to another object
 - Python uses it for testing equality



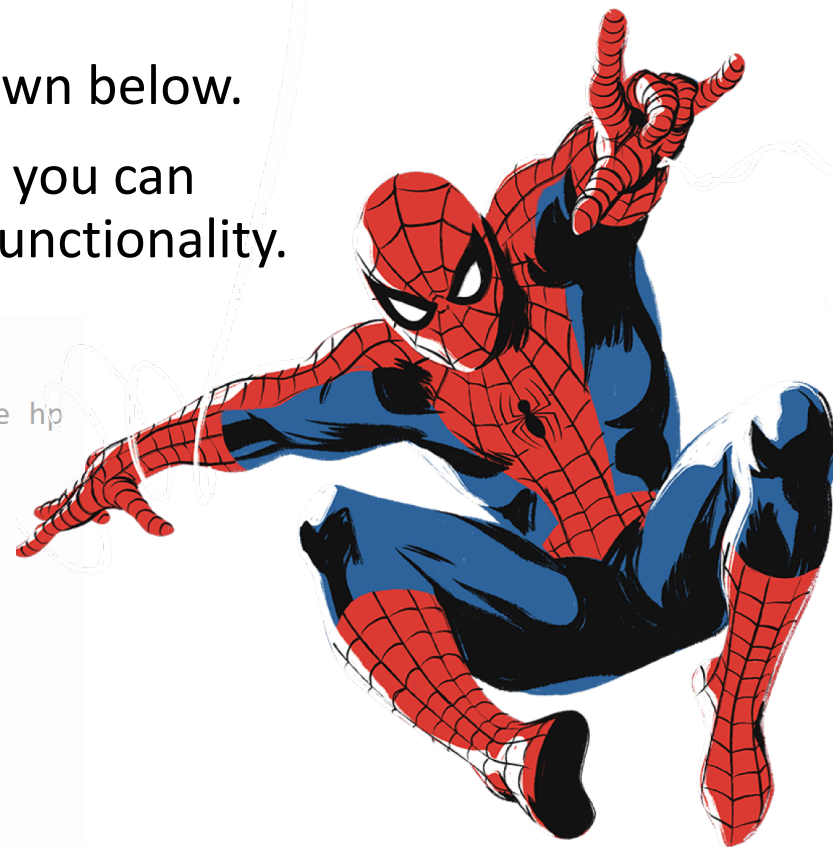
**Practice
Makes
Perfect**

Practice

Write the classes SpiderMan to pass the test cases shown below.

Do not hardcode against the testcase values, though you can assume the testcases and comments cover the needed functionality.

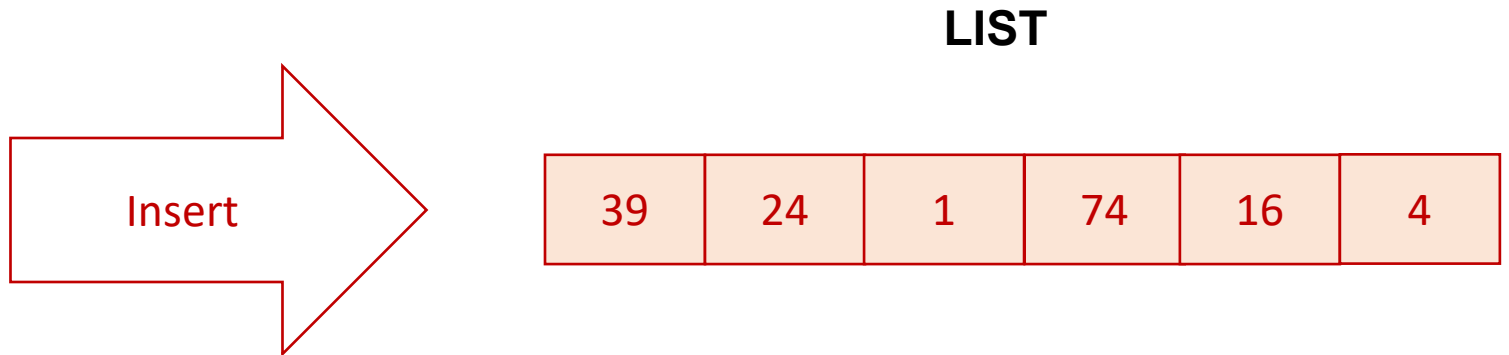
```
s = SpiderMan("Peter")
assert(str(s) == "SpiderMan(Peter) with 100 hp")
# SpiderMen can be equal to each other. This considers the name, but not the hp
assert(s == SpiderMan("Peter"))
assert(s != SpiderMan("Bob"))
assert(s != 42)
assert(s != "SpiderMan(Peter) with 100 hp")
# SpiderMan can shoot a web
assert(s.shootWeb() == "Peter shoots a web")
# SpiderMan can get hurt
assert(s.takeDamage(25) == "Peter gets hit for 25 hp")
assert(str(s) == "SpiderMan(Peter) with 75 hp")
# Current hp doesn't impact equivalence
assert(s == SpiderMan("Peter"))
```



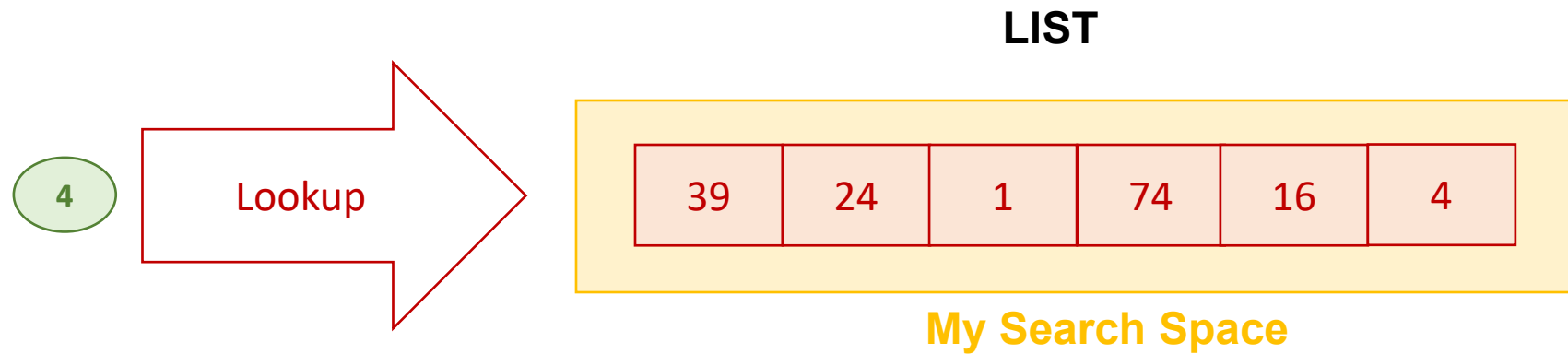
What special methods is the code expecting you to implement ?

- What is the expected behavior or outcome of these methods?

Inserting Elements in a List



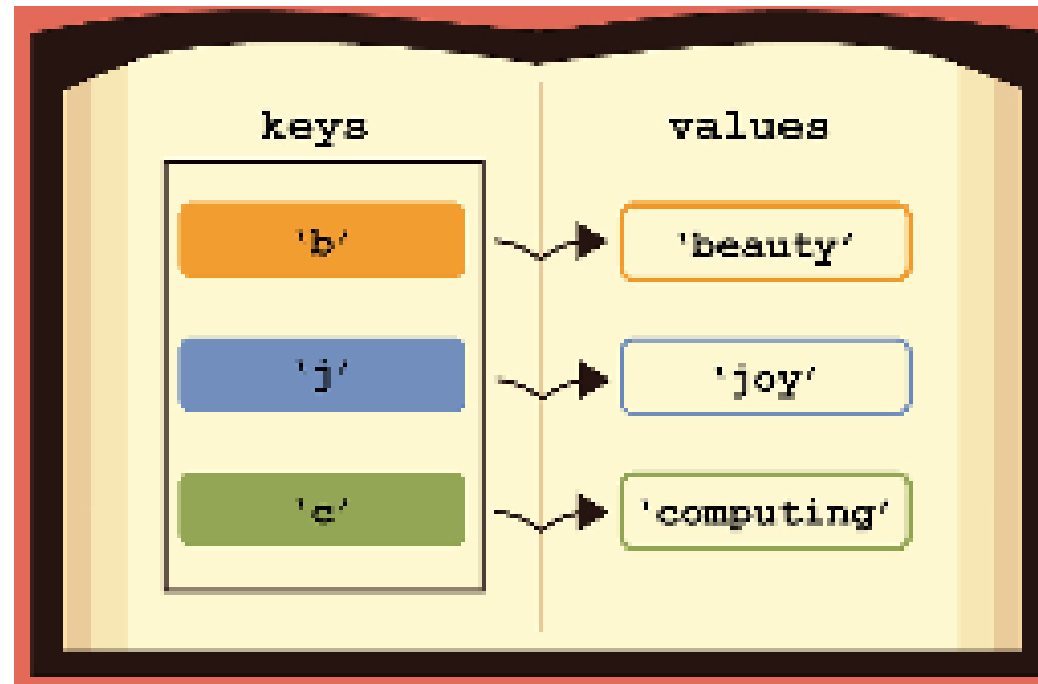
Looking Up an Element in a List



Equality Testing

Compare the value to each existing element in the list.
If it found, return it

Using Objects w/ Sets & Dictionaries



Inserting Elements in Sets & Hashing

Insertion Steps:

- 1) Hash (find the bucket)
- 2) Once in the bucket, perform Equality Testing
 - Compare the element to each existing element in the bucket
 - If it doesn't exist, add it



Hash Function

```
def myHash(n):  
    return n%10
```

BUCKETS

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

If I try to Insert a duplicate

Insertion Steps:

- 1) Hash (find the bucket)
- 2) Once in the bucket, perform Equality Testing
 - Compare the element to each existing element in the bucket
 - If it doesn't exist, add it

74

Insert

Hash Function

```
def myHash(n):  
    return n%10
```

BUCKETS

0	
1	1
2	
3	
4	24 74 4
5	
6	16
7	
8	
9	39

Looking Up an Element in a Set

Lookup Steps:

- 1) Hash (find the bucket)
- 2) Once in the bucket, perform Equality Testing
 - Compare the value to each existing element in the bucket
 - If found, return it

4

Lookup

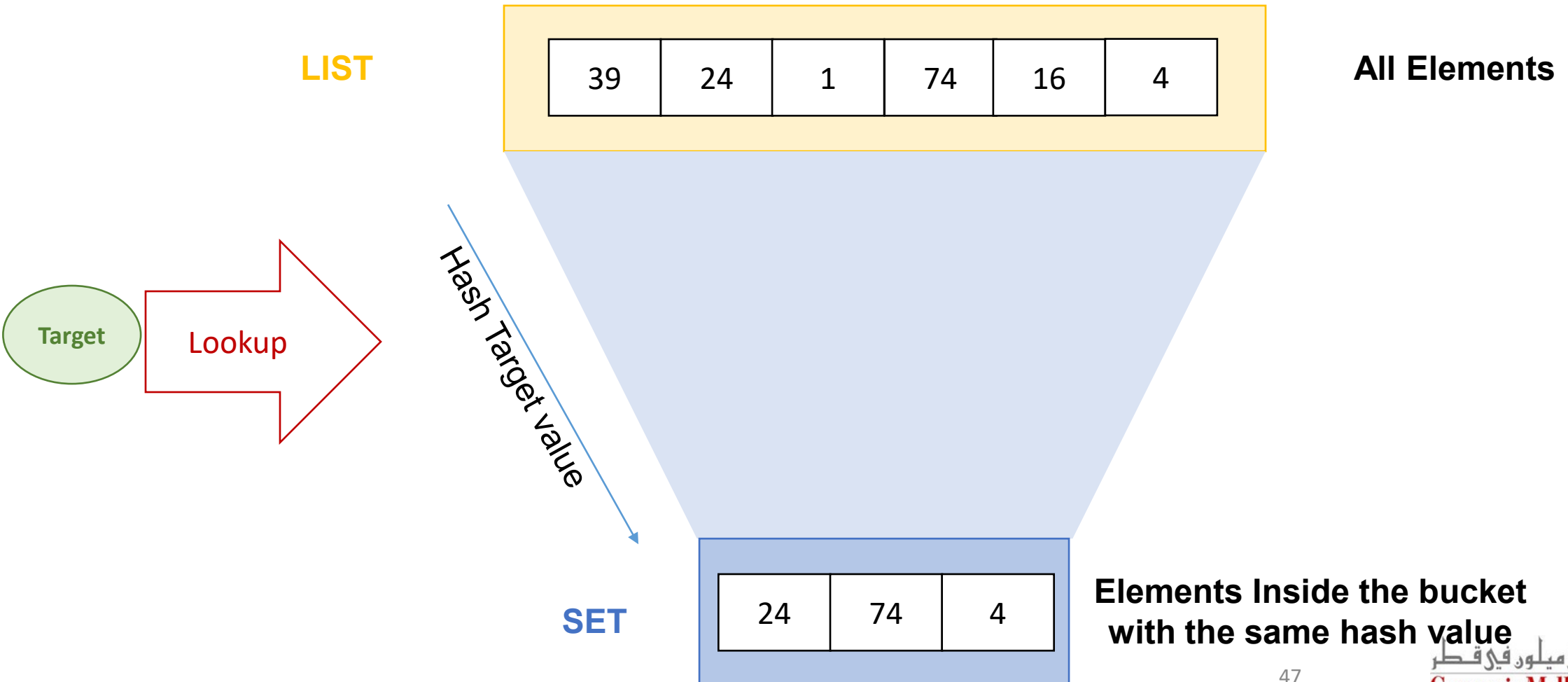
Hash Function

```
def myHash(n):  
    return n%10
```

BUCKETS

0	
1	1
2	
3	
4	24 74 4
5	
6	16
7	
8	
9	39

Looking Up Lists Vs Sets (Search Space)



Using in Sets & Dictionaries

```
1 class A(object):
2     def __init__(self, x):
3         self.x = x
4
5 s = set()
6 s.add(A(5))
7 print(A(5) in s) # False
8
9 d = dict()
10 d[A(5)] = 42
11 print(d[A(5)]) # crashes
```

Objects do not seem to hash right by default

```
5 a = A(5)
6 b = A(5)
7
8 print(hash(a) == hash(b)) # False
```

Using in Sets & Dictionaries

`__hash__` and `__eq__`

The `__hash__` method tells Python how to hash the object.

The properties you choose to hash on should be immutable types and should never change (so `hash(obj)` is immutable).

For sets and dictionaries to work properly, whenever you add hash, you need to add `eq` method

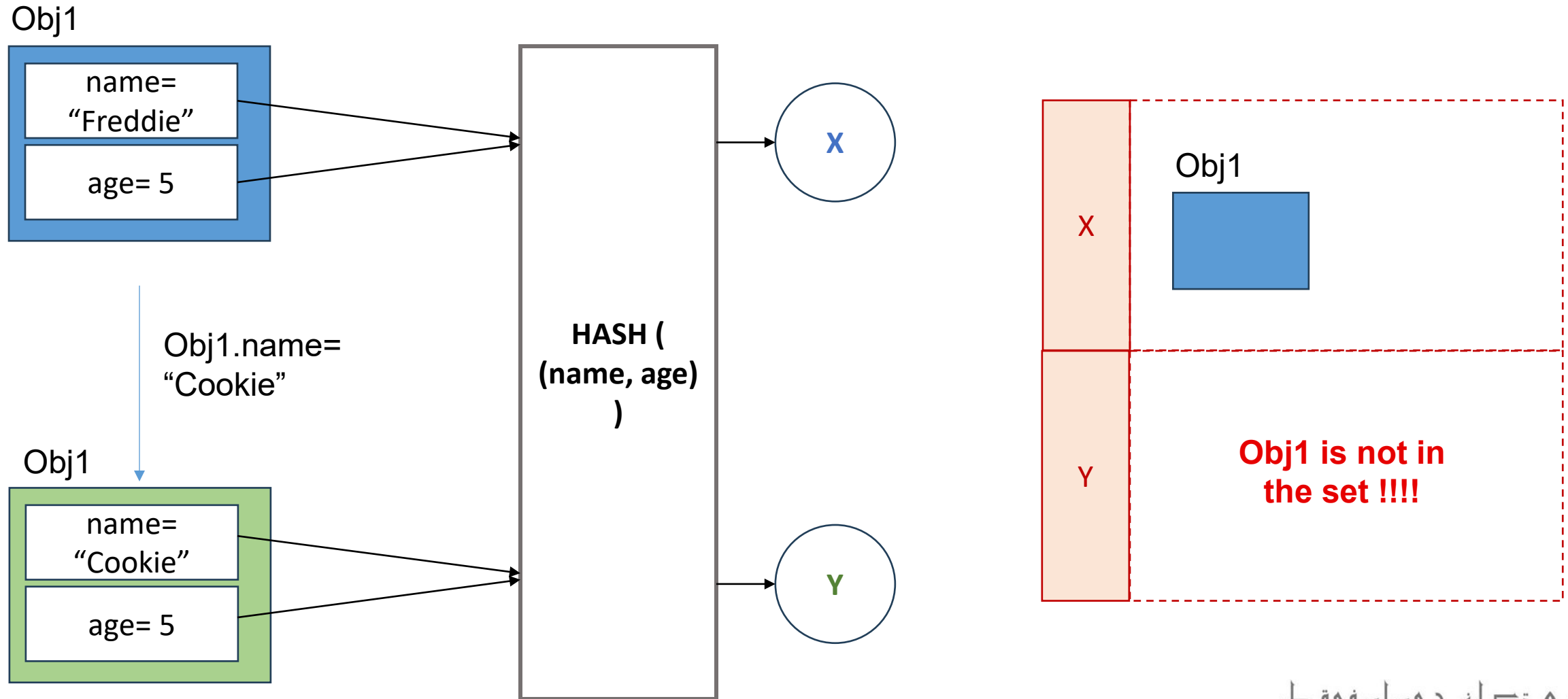
```
1 class A(object):
2     def __init__(self, x):
3         self.x = x
4     def __hash__(self):
5         return hash(self.x)
6     def __eq__(self, other):
7         return (isinstance(other, A) and (self.x == other.x))
8
9 s = set()
10 s.add(A(5))
11 print(A(5) in s) # True (whew!)
12
13 d = dict()
14 d[A(5)] = 42
15 print(d[A(5)]) # works!
```

Summary: Tips for Hashing in OOP

- **Should define both `__eq__` and `__hash__`**
 - Both should use the same attributes
 - If `__eq__` says that two objects are equal, they should have the same hash values
- **Attributes used should not change** throughout the program
 - Otherwise, you won't be able to locate objects later
- **Put attributes in a tuple**
- **Call `hash()` on the tuple**
 - Hash only works with immutable data types

Attributes you hash shouldn't change later in your code

Rules for Using Custom Objects w/ Sets



Objects & Aliases

Objects are mutable so aliases change!

Run this with the visualizer to make it clear!

```
from types import SimpleNamespace
import copy
```

```
dog1 = SimpleNamespace(name='Dino', age=10, breed='shepherd')
```

```
dog2 = dog1 # this is an alias
```

```
dog3 = copy.copy(dog1) # this is a copy, not an alias
```

```
dog1.name = 'Spot'
```

```
print(dog2.name) # Spot (the alias changed, since it is the same object)
```

```
print(dog3.name) # Dino (the copy did not change, since it is a different object)
```

Fundamentals of Programming & Computer Science CS 15-112

OOP – Part 2

Hend Gedawy

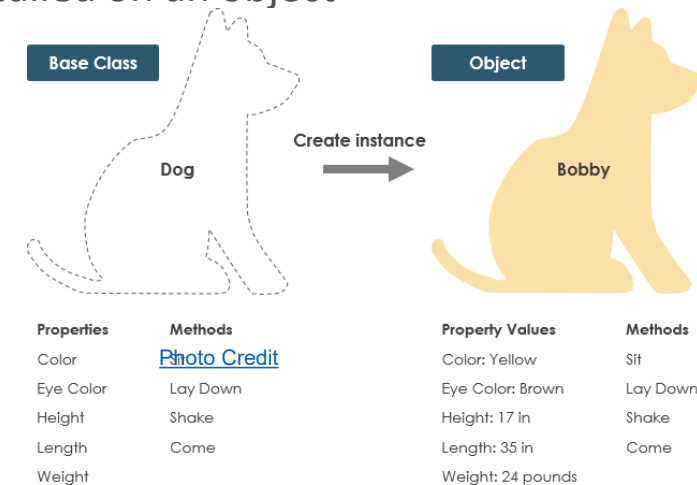
RECAP

1. We can define our own **special methods** to tell Python how instances behave in various ways, including:

1. `__init__` initializes an instance (setting its properties).
 - Called by default when a new instance is created
2. `__repr__` tells how to convert an instance to a string.
 - Called by default when `print()` or `str()` is called on an object
3. `__eq__` tells how two instances are equal.
 - Called by default when comparing two objects with `==` or `!=`
4. `__hash__` is required if we plan to use instances in a set or as keys in a dictionary.
 - Called by default when objects are used in sets or dicts or when the `hash()` is called on an object

2. Using **Objects with sets and dictionaries**

- Need to define `__hash__` and `__eq__` methods
- Need to use *same attributes* in both methods
- Only use attributes that will *not change* throughout the code



Outline

- **Class-Level Features**
 - Class Attributes
 - Static Methods
- **Inheritance**
 - Specifying A superclass
 - Overriding Methods
 - Isinstance vs type in inherited classes

Outline

- **Class-Level Features**

- Class Attributes
- Static Methods

- **Inheritance**

- Specifying A superclass
- Overriding Methods
- Isinstance vs type in inherited classes

Class Attributes

- A piece of data that belongs to an entire class rather than individual instances
 - It is **shared by all instances** of that class!
- Class attributes be **accessed from any instance** of that class
- Changing a class attribute value in one instance, changes it for every instance.

```
1 class A(object):
2     dirs = ["up", "down", "left", "right"]
3
4 # typically access class attributes directly via the class (no instance
5 print(A.dirs) # ['up', 'down', 'left', 'right']
6
7 # can also access via an instance:
8 a = A()
9 print(a.dirs)
10
11 # but there is only one shared value across all instances:
12 a1 = A()
13 a1.dirs.pop() # not a good idea
14 a2 = A()
15 print(a2.dirs) # ['up', 'down', 'left'] ('right' is gone from A.dirs)|
```

Static Methods

Static Methods in a class can be called directly without necessarily making and/or referencing a specific object.

```
1 class A(object):
2     @staticmethod
3     def f(x):
4         return 10*x
5
6 print(A.f(42)) # 420 (called A.f without creating an instance of A)
```

The `@staticmethod` decorator tells Python that the next method is not a normal method, but really just a normal function.

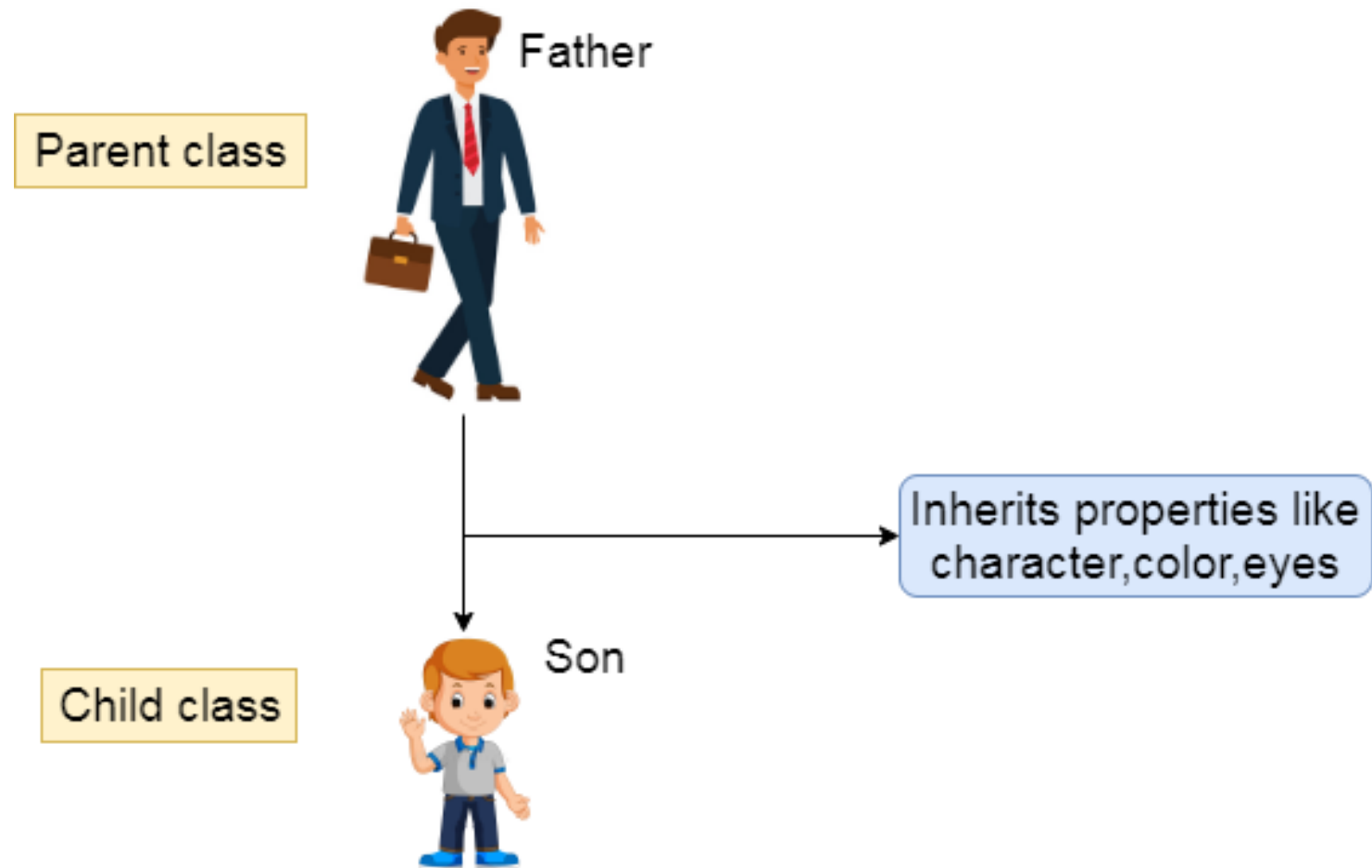
To call a static method, instead of using an instance before the dot, we use the class name.

Outline

- **Class-Level Features**
 - Class Attributes
 - Static Methods
- **Inheritance**
 - Specifying A superclass
 - Overriding Methods
 - Isinstance vs type in inherited classes

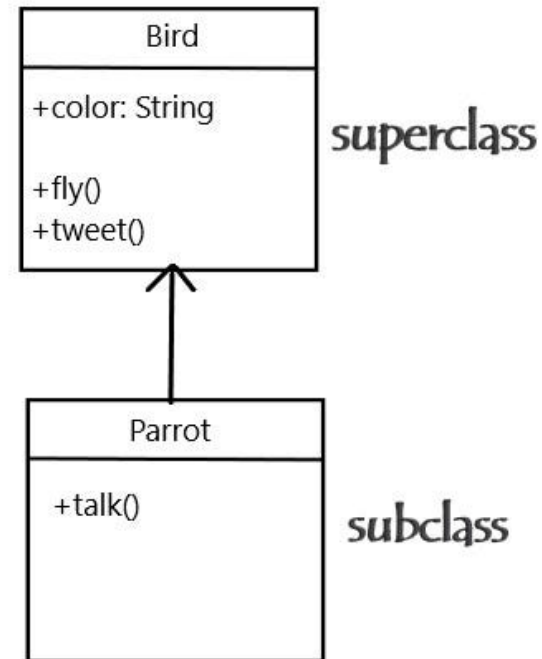
Inheritance

Defining Inheritance



Defining Inheritance

A **subclass** inherits all the methods from its superclass, and then can add or modify methods.



Specifying A Superclass

- We use **inheritance** when one class is a specialization of another class.
 - For example, a Cow is a special kind of FarmAnimal.
- We say that the specialized class is a **subclass**, and the more general class it inherits from is a **superclass**.
- When we create the subclass, we include the superclass in parentheses.
 - So we will create Cow(FarmAnimal). In this way, Cow instances are also FarmAnimal instances.
- Note that the `__init__` method of **Cow** called the **`__init__` method of its superclass** (that is, FarmAnimal) by using `super().__init__`.
- Notice that we defined `speak` in the FarmAnimal superclass, and then the subclasses (Horse and Cow) inherited this method.

```
1 class A(object):
2     def __init__(self, x):
3         self.x = x
4     def f(self):
5         return 10*self.x
6
7 class B(A):
8     def g(self):
9         return 1000*self.x
10
11 print(A(5).f()) # 50
12 print(B(7).g()) # 7000
13 print(B(7).f()) # 70 (class B inherits the method f from class A)
14 print(A(5).g()) # crashes (class A does not have a method g)
```

Overriding Methods

We can change a method's behavior in a subclass by overriding it.

```
1 class A(object):
2     def __init__(self, x):
3         self.x = x
4     def f(self):
5         return 10*self.x
6     def g(self):
7         return 100*self.x
8
9 class B(A):
10    def __init__(self, x=42, y=99):
11        super().__init__(x) # call overridden init!
12        self.y = y
13    def f(self):
14        return 1000*self.x
15    def g(self):
16        return (super().g(), self.y)
17
18 a = A(5)
19 b = B(7)
20 print(a.f()) # 50
21 print(a.g()) # 500
22 print(b.f()) # 7000
23 print(b.g()) # (700, 99)|
```

Isinstance VS type in inherited classes

```
1 class A(object): pass
2 class B(A): pass
3 a = A()
4 b = B()
5 print(type(a) == A) # True
6 print(type(b) == A) # False
7 print(type(a) == B) # False
8 print(type(b) == B) # True
9 print()
10 print(isinstance(a, A)) # True
11 print(isinstance(b, A)) # True (surprised?)
12 print(isinstance(a, B)) # False
13 print(isinstance(b, B)) # True
```

OOP CT – Exam Q

```
class A(object):
    def __init__(self):
        self.b = "sparkling"

    def f(self):
        return self.g()

    def g(self):
        return "apple"

    def h(self, s):
        print(f"{self.b} {s}")
        return self.f()

    def __repr__(self):
        return f'A {self.f()}'

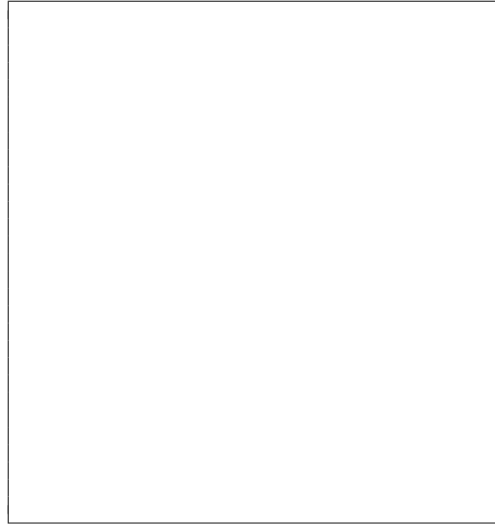
class B(A):
    def __init__(self, s):
        super().__init__()
        self.b = s

    def f(self):
        s=self.g()
        return f"pine{s}"

    def g(self):
        return super().g()

def ct3():
    b = B("still")
    print(b.h("water"))
    print(type(b) == A)
    print(isinstance(b, B))
    print(isinstance(b, A))
    return b

print(ct3())
```



(d) (4 points) CT4

```
class A(object):
    def __init__(self):
        self.b = "Panda"

    def f(self):
        return self.g()

    def g(self):
        return "KungFu"

class B(A):
    def __init__(self, s):
        super().__init__()
        self.b = s

    def f(self, s):
        self.d = " Master"
        return self.h(s) + self.d

    def h(self, s):
        print(f"{s} Soup")
        return super().g()

def ct4():
    a = A()
    b = B("Po")
    print(isinstance(b, A))
    print(b.b)
    return b.f("Noodles")

print(ct4())
```

OOP Advantages

1. Convenient

Instances give us a very convenient way to store all the properties of an object in one place.

2. Intuitive

Well-named classes with well-named methods and properties lead to very intuitive and clear code. For example, `dog.age` quite clearly refers to the age of the given dog instance.

3. Modular

Classes are **encapsulated**, meaning that they contain all the code that their instances require. Because of this, especially as our apps grow in size and complexity, OOP provides a natural way to organize our code.

4. Reusable

Well-designed classes lend themselves to code reuse. We can use the same classes in a variety of apps, which saves on programming time.

5. Adaptable

With the careful use of inheritance, we can adapt a class with a subclass that inherits the methods from the superclass, and only has to override the specific methods that should behave differently.

There are even more benefits, but from this list we see that OOP can be very helpful as we write larger and more complex apps.

Practice – Exam Question

Write the classes SpiderMan, TobeySpiderMan, and AndrewSpiderMan to pass the test cases shown below.

Do not hardcode against the testcase values, though you can assume the testcases and comments cover the needed functionality.
For full credit you must use inheritance appropriately and avoid unnecessary code duplication between classes

```
# TobeySpiderMan is a SpiderMan
t = TobeySpiderMan()
assert(isinstance(t, SpiderMan))
assert(str(t) == "SpiderMan(Tobey) with 100 hp")
# TobeySpiderMan has some special properties when taking damage
# Normally, he takes damage just like any other SpiderMan
assert(t.takeDamage(10) == "Tobey gets hit for 10 hp")
assert(str(t) == "SpiderMan(Tobey) with 90 hp")
# But, if he is wearing a Venom suit, then he only takes half damage
t.venomAttach()
assert(t.takeDamage(10) == "Tobey gets hit for 5 hp")
assert(str(t) == "SpiderMan(Tobey) with 85 hp")
# He can take off the Venom suit
assert(t.venomDetach() == True)
assert(t.takeDamage(10) == "Tobey gets hit for 10 hp")
assert(str(t) == "SpiderMan(Tobey) with 75 hp")
# But if he isn't wearing the Venom suit, then he can't take it off
assert(t.venomDetach() == False)
# Equivalence between TobeySpiderMan only considers the name, not the hp
assert(t == TobeySpiderMan())
assert(t != 42)
assert(t != "SpiderMan(Tobey) with 75 hp")
# But a TobeySpiderMan is never equal to a SpiderMan, even if the names are the same
assert(t != SpiderMan("Tobey"))
assert(SpiderMan("Tobey") != t)
# Other than this, he is a normal SpiderMan
assert(t.shootWeb() == "Tobey shoots a web")

# SpiderMen of all kinds can be put into a Set and retrieved later.
spiderSet = set()
spiderSet.add(s)
spiderSet.add(t)
spiderSet.add(a)
assert(SpiderMan("Peter") in spiderSet)
assert(TobeySpiderMan() in spiderSet)
assert(AndrewSpiderMan() in spiderSet)
```

```
# AndrewSpiderMan is a SpiderMan, but TobeySpiderMan and AndrewSpider are not
# instances of each other
a = AndrewSpiderMan()
assert(isinstance(a, SpiderMan))
assert(not isinstance(a, TobeySpiderMan))
assert(not isinstance(t, AndrewSpiderMan))
assert(str(a) == "SpiderMan(Andrew) with 100 hp")
# AndrewSpiderMan has a special property when shooting webs: He has a limited
# supply of web fluid. By default he can only shoot webs twice before needing
# to refill.
assert(a.shootWeb() == "Andrew shoots a web")
assert(a.shootWeb() == "Andrew shoots a web")
assert(a.shootWeb() == "Andrew is out of web fluid!")
a.refillWebFluid(3)
for i in range(3):
    assert(a.shootWeb() == "Andrew shoots a web")
assert(a.shootWeb() == "Andrew is out of web fluid!")
# When creating an AndrewSpiderMan, you can specify his initial web fluid capacity
a2 = AndrewSpiderMan(10)
for i in range(10):
    assert(a2.shootWeb() == "Andrew shoots a web")
assert(a2.shootWeb() == "Andrew is out of web fluid!")
# Equivalence between AndrewSpiderMan only considers the name, not the hp or webfluid
assert(a == AndrewSpiderMan())
assert(a2 == AndrewSpiderMan())
assert(a == a2)
assert(a != 42)
assert(a != "SpiderMan(Andrew) with 100 hp")
# But an AndrewSpiderMan is never equal to a SpiderMan, even if the names are the same
assert(a != SpiderMan("Andrew"))
assert(SpiderMan("Andrew") != a)
# Other than this, he is a normal SpiderMan
assert(a.takeDamage(25) == "Andrew gets hit for 25 hp")
assert(str(a) == "SpiderMan(Andrew) with 75 hp")
```



Write the classes `Cart` and `WishList` so that the test code runs as specified. Do not hardcode against the values used in the testcases, though you can assume the testcases cover the needed functionality. For full credit, you must use proper object-oriented design, including

6. (20 points) **Free Response: OOP**

Consider the following code designed to test two different classes: `Cart` and `WishList`:

```
# A (Shopping) Cart takes a string (not a list) of comma-separated items
# A cart stores items, and items are case insensitive
m1 = Cart('milk,eggs,MILK')
assert(m1.itemCount() == 2) # milk and eggs
# The .getItems method should return a set of unique items present in the cart
assert(m1.getItems() == {'milk', 'eggs'})
# Note that these items are lowercase. Ignore case and duplicates
m2 = Cart('Tomatoes,ONION,water,chips,CHIPS')
assert(m2.getItems() == {'tomatoes','onion','water','chips'})
assert(m2.itemCount() == 4) # Still just four items
assert(str(m2) == "Shopping Cart: 4 item(s)")
m3 = Cart('MILK')
assert(str(m3) == "Shopping Cart: 1 item(s)")
# You can add one new item at a time:
assert(m3.addItem('milk') == False) # Return False if the item is already present
assert(m3.addItem('eggs') == True) # Return True if we added a new item
assert(m3.itemCount() == 2)
assert(m3.getItems() == {'milk', 'eggs'})
assert((m3.getItems() == m1.getItems()) and (m3.getItems() != m2.getItems()))
# Once more, but with a new item:
assert(m3.addItem('water') == True) # True means the item was added!
# and so these all change:
assert(m3.itemCount() == 3)
assert(m3.getItems() == {'milk', 'eggs','water'})
# Lastly, all carts start out open
assert(m1.status == 'open')
# ...unless you pay them
m1.checkout()
assert(m1.status == 'closed')
m1.checkout()
assert(m1.status == 'closed') # still closed
# and you cannot add any more items
assert(m1.addItem('tomatoes') == False) # Return False if the cart has been checked out
assert(m1.status == 'closed') # still closed

# A Wish List is a shopping cart that it always remains open
w1 = WishList("LEMON")
assert(str(w1) == "Wish List: 1 item(s)")
assert(w1.itemCount() == 1)
assert(w1.getItems() == {'lemon'})
assert(w1.addItem("Salt") == True)
assert(w1.getItems() == {'lemon', 'salt'})
assert(w1.status == 'open')
w1.checkout() # does nothing, the wish list stays open
assert(w1.status == 'open')
```